# Symbolic Compression Protocols for Multilingual Multi-Agent Handoff Across Frontier Models: A Long Technical Report

Jordy Hernandez Cruzado
Hijos Del Sol Research
hernandezcruzadojordy@gmail.com

**Abstract**

Large language model agents increasingly need to transfer operational state across models, providers, and languages without carrying full conversational histories. We study compact symbolic formats as *state-transfer protocols*, asking when they preserve semantic fidelity, operational continuity, and handoff quality across heterogeneous model families. EXP05 is a broad discovery experiment: it compares natural, compressed, hybrid_min, and hybrid_state protocols over Spanish, English, and Chinese tasks using multiple frontier and semi-open generators, plus judge-drift, token-budget, Chinese compactness, and long-handoff mini-experiments. EXP06 is a controlled follow-up: it directly pairs compressed against hybrid_state, adds a Chinese tokenizer/variant ablation, and evaluates every valid generation with three judges. EXP07 and EXP08 add real-agent validation layers using LangGraph and OpenFang with objective handoff metrics; EXP08 scales this setting to 900 operational executions. EXP09 then connects protocols to deterministic file-level tool use over controlled repository tasks, EXP10–EXP12 use the public research artifact as controlled repository/page-maintenance surfaces, and EXP13 introduces a role-conditioned multi-agent scientific-repository benchmark. Across EXP05, compressed is the strongest general baseline, while hybrid_state significantly improves paired state preservation in the principal subset ($\Delta = +0.133$, 95% CI $[+0.025, +0.240]$). EXP06 confirms the state-preservation advantage under stricter controls ($\Delta = +0.066$, 95% CI $[+0.032, +0.100]$), but also shows that hybrid_state does not win global quality ($\Delta Q = -0.208$, 95% CI $[-0.239, -0.177]$). EXP07 and EXP08 confirm that these protocols can be evaluated in real agent surfaces without relying solely on LLM judges, but neither shows a global hybrid_state win: compressed remains the strongest operational baseline, while hybrid_state shows only conditional advantages in selected state/constraint strata. EXP09–EXP13 show that protocol/tool interfaces can support deterministic file-level and public-repository task completion under objective validators, after interface-level repair of action schemas, parser contracts, and reasoning-budget behavior. EXP13 is more discriminative than the saturated public-artifact tests: compressed completed 64/64 real cells, while hybrid_state completed 62/64 and retained two final contract failures. We therefore argue that compressed is the universal baseline, while hybrid_state is a specialized protocol whose benefits are strongest for explicit state variables, not aggregate quality. Chinese and tokenizer-ablation results suggest language representation matters, but the mechanism remains only partially isolated.

## 1 Introduction

Autonomous and semi-autonomous language-model agents often need to continue work after context truncation, model substitution, provider failure, or handoff to another agent. The usual solution is to retain more natural-language context. This is expensive and fragile: longer context can increase cost, latency, and ambiguity, while still failing to preserve the operational state that matters for continuation.

This paper studies a different framing: compressed and symbolic prompt formats as *state-transfer protocols*. Rather than asking only whether a prompt can be shortened, we ask whether compact formats preserve the information required for another model to continue a task. This distinction matters because a protocol can be token-efficient yet operationally weak, or slightly less natural yet better at carrying state.

The empirical design has nine stages. EXP05, named `EXP05_PREMIUM_MULTIMODEL_TRILINGUAL`, is a broad discovery experiment across modes, languages, model families, judges, and stress tests. EXP06, named `EXP06_CAUSAL_PROTOCOL_TRANSFER`, is a narrower confirmation experiment that directly tests the strongest EXP05 claim: hybrid_state should improve operational state transfer even if it does not maximize global quality. EXP07, named `EXP07_REAL_AGENT_HANDOFF_OBJECTIVE_METRICS`, asks whether the same protocol family can be evaluated inside real agent frameworks using objective continuation metrics rather than only LLM judges. EXP08, named `EXP08_REAL_AGENT_SCALEUP`, scales the real-agent evaluation to 30 tasks, 5 repetitions, and 900 operational executions. EXP09, named `EXP09_REAL_TOOL_USE_AGENT_TASKS`, moves from handoff scoring to controlled repository-like tool use with deterministic validators. EXP10–EXP12 use the public landing/repository artifact as controlled culmination and page/conflict-maintenance tasks. EXP13, named `EXP13_REAL_MULTI_AGENT_SCIENT` is the main role-conditioned scientific-repository benchmark. The later tool-agent experiments are intentionally narrower interface validations rather than broad multi-model comparisons.

The resulting story is deliberately not a winner-takes-all ranking. Compressed text remains the strongest universal baseline. Hybrid_state is weaker on aggregate quality, but it repeatedly improves state preservation in judge-based experiments, the metric most aligned with multi-agent continuation. EXP07 and EXP08 further show that real-agent objective metrics are feasible and useful, while also warning against overgeneralization: hybrid_state does not dominate real-agent execution globally. EXP09 and EXP10 add a practical interface lesson: tool-use success depends not only on the protocol text, but also on the action schema, validator contract, parser tolerance, dependency tracking, and model reasoning budget. This distinction is central: compact protocols should be optimized against their intended transfer function and execution surface, not only against a single global score.

## 2 Related Work

This work connects several lines of research: prompt compression, agent memory, state summarization, tool-use evaluation, multilingual evaluation, and LLM-as-judge methodology.

**Prompt compression.** Prompt compression studies attempt to reduce token count while preserving task utility. These approaches usually optimize for shorter prompts, shorter context, cheaper inference, or equivalent downstream answers. Our framing is adjacent but different: the compressed artifact is not only a shorter prompt, but a communication protocol between agents. This shifts the evaluation target from response quality alone to state preservation, continuity, recoverability, and protocol transfer.

**Agent memory and state summarization.** LLM-agent systems often use summaries, scratchpads, memory stores, retrieval, or structured state objects to survive long tasks. Natural-language summaries are flexible but can blur variables, constraints, and pending actions. Structured memories can preserve fields more explicitly, but they require schemas and update discipline. The hybrid_state protocol sits between these traditions: it is compact enough to function as a handoff note, but explicit enough to carry operational state variables.

**Multi-agent handoff and planning.** Agent frameworks increasingly expose handoff, planning, tool routing, and workflow state as first-class concerns. Prior work often evaluates final task

success or qualitative continuation. EXP07 and EXP08 instead measure variable recovery, subtask completion, plan continuity, constraint retention, handoff success, and state-error count. This is meant to separate "the next agent wrote something plausible" from "the next agent preserved the operational state needed to continue."

**Tool-use evaluation.** Tool-using agents introduce a second interface problem: the model must satisfy both the natural-language task and a machine-readable action contract. EXP09 and EXP10 therefore evaluate deterministic validators, tool schemas, parser repair, and historical interface failures. These experiments are closer to protocol/interface validation than to broad model benchmarking.

**Multilingual and tokenization effects.** Multilingual evaluation shows that model behavior varies by language, script, tokenization, training distribution, and cultural-linguistic structure. EXP05 directly compares Spanish, English, and Chinese, while EXP06 adds native ZH, ZH_PINYIN, EN_LITERAL, and ES_LITERAL variants. This design does not fully isolate tokenization, but it tests whether compact symbolic formats remain stable across representation regimes.

**LLM-as-judge and judge drift.** LLM-as-judge methodology is common for scalable evaluation, but evaluator identity can shift score levels and relative rankings. This paper treats judge scores as useful but not sufficient: EXP06 uses three-judge calibration, MiniEXP-B measures judge drift, and EXP07–EXP10 add objective validators where possible.

# 3 Method

## 3.1 Communication Modes

We compare four protocol modes.

**natural** uses standard human-readable prose.

**compressed** uses a compact baseline similar to dense operational notes.

**hybrid_min** uses the most aggressively minimized hybrid symbolic form.

**hybrid_state** emphasizes explicit state preservation, continuity markers, and handoff-relevant fields.

## 3.2 Languages and Tasks

The principal experiment uses Spanish (ES), English (EN), and Chinese (ZH). Tasks cover contextual summarization, handoff, planning, persistent operational state, and multilingual transfer. The principal design contains 15 task templates, 3 languages, 4 modes, 2 repetitions, and 5 generator families, partitioned into resumable execution parts.

## 3.3 Models

The principal generator pool includes Grok 4.20 Non-Reasoning, Gemini 3.1 Pro Preview, Llama 4 Maverick, Qwen3-Next-80B, and GLM 5. Additional controlled extensions include Azure GPT instant, Grok reasoning/non-reasoning comparisons, Azure Grok 4.3, judge-drift models, token-budget stress models, and long-handoff models.

## 3.4 Evaluation

Evaluators return strict JSON metrics. Positive metrics are scored with higher values better: semantic fidelity, clarity, utility, completeness, state preservation, operational continuity, context recover-

ability, handoff quality, compactness, language stability, and inter-model transferability. Negative metrics are scored with lower values better: ambiguity and information loss.

We compute a descriptive quality index:

$$Q = \frac{\text{mean(positive)} + (6 - \text{mean(negative)})}{2}.$$

This index is used for ranking, but individual metrics are reported separately because the main scientific claims concern state preservation, continuity, compactness, and language behavior.

### 3.5 Data Cleaning

All analyses use cleaned and deduplicated outputs. The cleaning pass keeps the latest successful generation/evaluation per cell, removes superseded retries, preserves unique MiniEXP-C generation failures as experimental evidence, and verifies that no evaluation points to a missing generation. EXP06 additionally preserves provider policy failures as terminal events: 21 Azure GPT instant generation cells were marked `policy_failure`, retained for provider-reliability analysis, and excluded from score aggregation because no output existed to evaluate. EXP07 preserves operational startup/calibration errors separately, then scores the latest successful execution per real-agent cell. EXP08 reports two views: an *operational-complete* view with 900/900 latest cells marked OK, and a stricter *JSON-valid* view with 824/900 cells after excluding blank-output responses from Azure GPT 5.4 High. EXP09 keeps the latest successful row per tool-use cell, produces `clean_latest_ok.jsonl`, and preserves 155 historical repaired errors separately rather than deleting them. EXP10 follows the same policy: the latest-cell view contains 80/80 successful cells, including 64/64 Gemini 3.5 Flash tool-agent cells, while 17 historical interface errors are retained as repaired evidence.

## 4 Expanded Methodology

This section expands the experimental method because the main reproducibility risk in this work is not the computation of the tables, but the exact interface between protocol text, model route, evaluator contract, cleaning policy, and resumed execution. The same nominal prompt can behave differently if a provider silently changes routing, if an evaluator permits non-JSON prose, if a retry overwrites a failed cell without a retained event log, or if a tool agent is allowed to touch files outside scope. The methodology therefore treats the protocol as only one part of a larger experimental surface.

### 4.1 Methodological Objective

The research question is not whether terse prompts can produce pleasant summaries. The question is whether compact communication formats can function as operational state-transfer protocols across model families, languages, judges, and agent surfaces. A successful protocol must preserve three different classes of information. First, it must preserve semantic content: what happened and what claims are supported. Second, it must preserve operational state: variables, constraints, pending subtasks, dependencies, and next actions. Third, it must preserve interface compliance: the receiving model or agent must return parseable JSON, valid tool calls, or file edits that pass deterministic validators.

This distinction explains why the paper reports several metrics instead of a single aggregate score. A protocol can win compactness while losing fidelity. It can improve state preservation while

losing general quality. It can pass judge-based evaluation but fail in a tool interface because the output violates a parser contract. The design therefore separates discovery, controlled confirmation, objective handoff, and deterministic tool-use validation.

### 4.2 Protocol Design Principles

The four modes are intentionally ordered by structure rather than by expected quality. `natural` is the human baseline. It maximizes ordinary readability but often leaves state implicit. `compressed` is the main operational baseline. It uses short field-like clauses while remaining flexible and human-readable. `hybrid_min` is an extreme compression condition designed to test whether minimum symbolic density survives across models. `hybrid_state` is the state-oriented protocol. It sacrifices some naturalness to expose variables, constraints, evidence, plan steps, and continuation markers.

The protocols are not intended as universal grammars. They are experimental formats that make different tradeoffs. In particular, `hybrid_state` is not expected to improve every metric. Its purpose is narrower: make state variables and constraints harder for the next model to miss. This matters because many handoff failures are not hallucinations in the usual sense. They are state-decay failures: the next agent writes a plausible answer while forgetting a constraint, mutating a claim, skipping a dependency, or continuing the wrong plan.

### 4.3 Task-Bank Construction

Task templates were built around operational continuation rather than isolated question answering. The principal EXP05 bank contains contextual summarization, agent handoff, planning, persistent state, and multilingual transfer tasks. Each template is instantiated across Spanish, English, and Chinese. The controlled EXP06 bank keeps the same conceptual target but narrows the comparison to `compressed` versus `hybrid_state`. EXP07 and EXP08 convert the handoff idea into objective continuation tasks. EXP09 converts the idea into file-level tool use. EXP10 converts it into public repository maintenance.

The task bank is intentionally not a benchmark of general intelligence. It is a benchmark of state transfer. A model can be broadly capable and still lose information under a compressed protocol. Conversely, a compact protocol can pass a targeted state check without being globally superior. This is why the same paper can report that `hybrid_state` improves state preservation in EXP05/EXP06 while `compressed` remains the safer operational baseline in EXP07/EXP08.

### 4.4 Language and Representation Controls

EXP05 uses ES, EN, and ZH to test whether compact protocols generalize across languages and scripts. EXP06 adds ZH_PINYIN, ES_LITERAL, and EN_LITERAL variants to separate language content from representation format. The design does not fully isolate tokenizer mechanics, but it creates a sharper contrast than a simple multilingual average. Native ZH and romanized ZH_PINYIN contain related semantic content but different surface representation. Literal variants stress whether rigid translated structure changes protocol behavior.

The correct interpretation is cautious. Stronger ZH results support a representation hypothesis, not a closed causal proof. Possible mechanisms include script density, tokenizer segmentation, pretraining distribution, model familiarity with compact Chinese structures, and prompt-format interaction. Future work should add tokenizer-level counts, controlled synthetic strings, and provider-independent tokenization audits.

### 4.5 Model Routing and Provider Surfaces

The experiments use provider-facing model identifiers as observed at execution time. This is important because modern hosted models are not static objects. Names, routes, quotas, safety filters, context policies, and output defaults can change. The paper therefore treats model route as part of the experimental condition. EXP05 includes multiple generator families. EXP06 includes separate judge routes. EXP07 and EXP08 include real-agent execution surfaces and model differences. EXP09 and EXP10 narrow the provider surface to test deterministic tool interfaces.

Provider-specific failures are not discarded as irrelevant noise. Quota exhaustion, policy failure, blank output, JSON truncation, and action-schema mismatch are operationally meaningful when the research target is agent handoff. A protocol that only works when all infrastructure is perfect is not yet a robust handoff protocol.

### 4.6 Resumable Execution

Every large experiment is designed as resumable execution. A cell is identified by phase, task, language or variant, mode, generator, judge when applicable, framework when applicable, and repetition. The runner appends events instead of mutating history. A later successful cell can supersede an earlier parse error, quota event, or retry failure, but the historical event remains available for audit.

This matters for two reasons. First, provider quotas and long windows make uninterrupted execution unrealistic. Second, silent overwrites would make the final dataset look cleaner than the actual experiment. The latest-cell analysis is the scoring view; the historical log is the systems-evidence view. EXP09–EXP12 are especially dependent on this distinction because their final success rates are saturated after interface repair, while their historical errors reveal the real engineering lessons. EXP13 keeps the same policy but retains two final latest-cell failures as benchmark evidence rather than repairing them away.

### 4.7 Rate Limits and Retry Policy

The execution policy is conservative. When a provider returns quota exhaustion, the runner waits before retrying. The target for rate-limited Gemini evaluation was kept around four calls per minute during constrained phases. When a failure appears to come from a longer quota window rather than a local bug, the process pauses rather than switching judges silently. The rule is methodological: do not trade quota failures for hidden evaluator substitution unless the substitution is explicitly marked.

Retries are not counted as independent scientific observations. They are execution events. The analysis keeps the last successful output per cell for scored comparisons and separately preserves unresolved or repaired failures. This prevents retry-heavy phases from inflating sample size.

### 4.8 Evaluation Prompt Contract

Judge-based experiments require strict JSON. The evaluator prompt instructs the judge to return numeric metrics and no prose outside JSON. Parse failures are therefore meaningful. A parse failure can indicate model noncompliance, token-budget pressure, prompt ambiguity, or provider truncation. EXP06 and EXP08 both show why this matters: strict JSON validity can differ from operational completion.

The judge is not treated as ground truth. It is a measurement instrument. Therefore the paper reports judge identity, judge calibration, and judge drift. EXP06 uses three judge routes to

reduce dependence on one evaluator. MiniEXP-B directly tests judge drift. The later real-agent experiments reduce judge dependence by using objective validators.

### 4.9 Objective Validator Contract

EXP07 and EXP08 use objective continuation metrics. The validator checks whether required variables are recovered, subtasks are completed, plan continuity is retained, constraints remain intact, handoff succeeds, and state errors occur. These metrics are narrower than semantic quality but more directly operational. A fluent answer that fails to preserve a variable should not receive full handoff credit.

EXP09 and EXP10 extend objective validation to tool use. The validator checks deterministic file-level results, JSON validity, expected file changes, absence of scope drift, dependency preservation, recovery, and claim preservation. This moves the evaluation from "does a judge like the answer?" toward "did the repository state end correctly?"

### 4.10 Cleaning and Deduplication

Cleaning uses a latest-cell policy. For each cell identifier, the final successful record is the scored record. Historical errors remain in separate logs. Terminal policy failures remain terminal when no valid replacement exists. Superseded parse errors, quota events, and repaired interface failures are not deleted; they are excluded from aggregate scoring but retained for reproducibility.

This is why the paper reports both clean scored results and repaired-error counts. A final 288/288 or 80/80 success rate is not the whole story if the path to that state involved schema repair. The repaired errors are not embarrassments; they are part of the systems result.

### 4.11 Cost and Token Ledger

Each call is logged with model, provider, stage, estimated input tokens, estimated output tokens, reasoning tokens when available, estimated cost, and remaining budget when tracked. Provider dashboards can provide delayed billing truth, but experimental analysis needs immediate per-call estimates. The ledger is therefore an experimental instrument, not a replacement for final billing export.

The cost ledger is used in three ways. First, it protects the budget during long runs. Second, it allows cost/quality plots. Third, it records provider failure modes that correlate with output length, reasoning budget, or model tier. Cost is not only accounting; it is part of the operational profile of a protocol.

### 4.12 Statistical Procedure

The main statistical unit is a paired cell where `compressed` and `hybrid_state` share the same task, language or variant, generator, judge, and repetition. Paired deltas are computed as candidate minus baseline. Confidence intervals are computed over paired deltas. The descriptive quality index is used to summarize aggregate quality, but the paper's strongest claims rely on specific metrics such as state preservation, operational continuity, ambiguity, information loss, and objective recovery.

The current analysis is sufficient for a technical report but not the final word. A venue-strength version should add clustered bootstrap or mixed-effect models with random effects over task, language, generator, judge, framework, and repetition. This is declared as a limitation rather than hidden.

### 4.13 Threat Model for Claims

The paper distinguishes strong claims, bounded claims, exploratory claims, and non-claims. Strong claims require convergence across experiments or strict paired evidence. Bounded claims are supported only under specific surfaces. Exploratory claims identify signals that need stronger controls. Non-claims are explicitly blocked. For example, the paper does not claim that `hybrid_state` is globally better, that ZH proves tokenizer causality, or that model rankings measure general intelligence.

### 4.14 Why Methodology Is Long

The methodology is long because the object of study is not a standalone model output. It is a protocol moving through a full system: prompt, provider, model, evaluator, parser, validator, retry policy, cleaning rule, and cost ledger. In this kind of work, many false discoveries come from invisible interfaces. Making those interfaces explicit is part of the contribution.

## 5 Experiments

### 5.1 Principal EXP05

The principal experiment evaluates the four communication modes across ES/EN/ZH and five generator families. The cleaned principal subset contains 1,800 evaluations.

### 5.2 MiniEXP-A: Grok Reasoning

MiniEXP-A compares Grok 4.20 Non-Reasoning and Grok 4.20 Reasoning on matched compact-protocol tasks. A2 separately evaluates Azure Grok 4.3 and is not pooled with Grok 4.20.

### 5.3 MiniEXP-B: Judge Drift

MiniEXP-B re-evaluates the same 120 source outputs with four judges: Gemini 3.1 Pro Preview, Gemini 3.5 Flash, Azure GPT 5.4 High, and Grok 4.20 Reasoning.

### 5.4 MiniEXP-C: Token Budget Stress Test

MiniEXP-C evaluates how models respond to reduced output budgets. Generation failures are not discarded when no successful retry exists, because they indicate stress-test failure.

### 5.5 MiniEXP-D: Chinese Compactness

MiniEXP-D isolates Chinese compact protocol behavior to test whether ZH performance reflects a real language/tokenization signal or only principal-experiment noise.

### 5.6 MiniEXP-E: Long Handoff

MiniEXP-E studies multi-turn handoff durability under longer operational chains.

### 5.7 EXP06: Controlled Causal Follow-up

EXP06 tests the strongest EXP05 interpretation under stricter controls. EXP06-A directly pairs compressed and hybrid_state on causal handoff tasks across ES, EN, and ZH. EXP06-B probes the language/tokenization hypothesis using ZH, ZH_PINYIN, EN_LITERAL, and ES_LITERAL variants. EXP06-C evaluates every valid generation with three independent judges: Gemini 3.5 Flash, Azure GPT 5.4 High, and Grok 4.20 Reasoning. The cleaned EXP06 dataset contains 1,563 valid generations and 4,689 successful evaluations.

### 5.8 EXP07: Real-Agent Handoff

EXP07 evaluates the same protocol family inside real agent surfaces. It uses LangGraph StateGraph and OpenFang CLI/daemon runs over 10 operational handoff tasks, 3 modes, and 3 repetitions. LangGraph is run with Azure GPT chat latest and Azure GPT 5.4 High; OpenFang is run with Azure GPT chat latest. The cleaned EXP07 dataset contains 270 successful real-agent executions after deduplication, with 4 operational calibration errors retained separately. Unlike EXP05 and EXP06, EXP07 uses objective schema metrics: variable recovery, subtask completion, plan continuity, constraint retention, handoff success, and state-error count.

### 5.9 EXP08: Real-Agent Scale-Up

EXP08 scales the real-agent design to 30 operational tasks, 2 modes, 5 repetitions, and 900 operational executions. LangGraph is run with Azure GPT chat latest and Azure GPT 5.4 High; OpenFang is run with Azure GPT chat latest. The goal is not to introduce a new protocol, but to test whether EXP07's real-agent conclusions survive a larger matrix. The final operational view contains 900/900 OK cells and zero latest operational errors. The stricter JSON-valid view contains 824/900 cells; the remaining 76 are blank-output responses from Azure GPT 5.4 High under LangGraph, concentrated more heavily in hybrid_state.

### 5.10 EXP09: Real Tool-Use Agent Tasks

EXP09 evaluates whether compact protocols can be connected to real file-level tool use. It uses 24 controlled repository-like tasks, 2 modes, 3 repetitions, and 2 routes: a deterministic reference tool agent and Gemini 3.5 Flash through Vertex. Task families include file reading, CSV transformation, JSONL repair, Markdown table generation, and checksum manifest creation. Each cell runs in an isolated workspace and is scored by deterministic validators, not LLM judges. The cleaned dataset contains 288/288 latest successful cells.

### 5.11 EXP10: Public Repo Tool-Agent Culmination

EXP10 evaluates the same protocol/tool interface on the public-facing research artifact: a landing/repository package explaining the experimental program from EXP01 through EXP10. It uses 16 controlled repository-maintenance tasks, 2 protocol modes, 2 repetitions, Gemini 3.5 Flash through Vertex, and a deterministic local scaffold route. The validators measure task success, claim drift, scope drift, dependency preservation, recovery after handoff, constraint preservation, and absence of secret leakage. The security check scans generated public artifacts for API-key-like strings, bearer-token patterns, credential filenames, and forbidden raw secret exposure. The final latest-cell view contains 64/64 Gemini tool-agent cells and 16/16 dry-local scaffold cells.

# 6 Results

## 6.1 Principal Mode Results

Table 1 shows the principal experiment ranking by mode. Compressed achieves the highest quality index. Hybrid_state ranks second and is strongest on state preservation.

| Mode | $n$ | $Q$ | Fidelity | State | Continuity | Compact | InfoLoss |
|---|---|---|---|---|---|---|---|
| compressed | 450 | 4.301 | 4.351 | 4.233 | 4.327 | 4.833 | 1.949 |
| hybrid_state | 450 | 4.180 | 4.189 | 4.369 | 4.320 | 4.807 | 2.022 |
| natural | 450 | 4.161 | 4.184 | 3.991 | 4.024 | 4.409 | 2.033 |
| hybrid_min | 450 | 3.769 | 3.853 | 3.840 | 3.742 | 4.849 | 2.467 |

Table 1: Principal EXP05 results by communication mode. Negative metrics such as information loss are better when lower.

## 6.2 Paired Effects Against Compressed

Table 2 reports paired deltas against compressed. Hybrid_state improves state preservation by $+0.133$ with 95% CI $[+0.025, +0.240]$. Its operational continuity is effectively tied with compressed. Hybrid_min preserves compactness but loses quality, fidelity, state, and continuity.

| Metric | Mode | $n$ | $\Delta$ | 95% CI Low | 95% CI High |
|---|---|---|---|---|---|
| quality index | hybrid_state | 430 | -0.108 | -0.203 | -0.012 |
| semantic fidelity | hybrid_state | 430 | -0.142 | -0.255 | -0.029 |
| state preservation | hybrid_state | 430 | +0.133 | +0.025 | +0.240 |
| operational continuity | hybrid_state | 430 | +0.007 | -0.099 | +0.113 |
| compactness | hybrid_state | 430 | -0.014 | -0.080 | +0.052 |

Table 2: Paired deltas for hybrid_state against compressed in the principal experiment.

## 6.3 Language Effects

Table 3 shows a consistent ZH advantage in the principal subset. This supports the hypothesis that language structure and tokenization may affect compact protocol performance, but it does not yet isolate the causal mechanism.

| Language | $n$ | $Q$ | Fidelity | State | Continuity |
|---|---|---|---|---|---|
| ZH | 600 | 4.180 | 4.207 | 4.197 | 4.180 |
| EN | 600 | 4.108 | 4.143 | 4.115 | 4.108 |
| ES | 600 | 4.020 | 4.083 | 4.013 | 4.022 |

Table 3: Principal EXP05 results by language.

## 6.4 Generator Effects

Grok 4.20 Non-Reasoning leads the principal generator ranking ($Q = 4.466$), followed by GLM 5 ($Q = 4.394$), Qwen3-Next-80B ($Q = 4.248$), Llama 4 Maverick ($Q = 3.874$), and Gemini 3.1 Pro

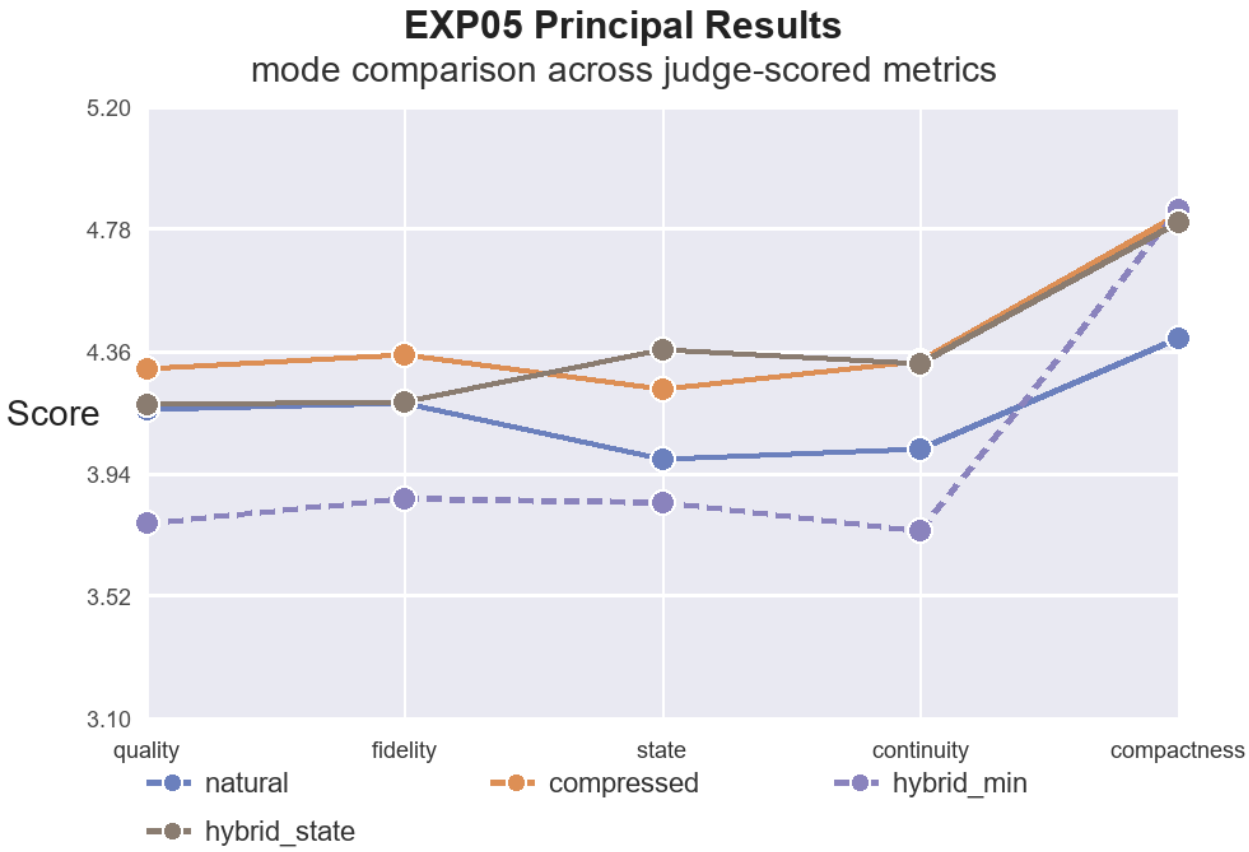


Figure 1: EXP05 principal mode tradeoff across quality index, state preservation, and compactness.

Preview ($Q = 3.532$). This ranking should be interpreted as protocol/task behavior under this setup, not as a general benchmark of model intelligence.

### 6.5 Judge Drift

Judge-drift results show strong but imperfect agreement. Gemini 3.1 and Gemini 3.5 correlate most strongly ($r = 0.839$). Azure GPT 5.4 High is consistently stricter than Gemini and Grok judges, with mean quality-index deltas around $-0.95$ versus Gemini and $-0.43$ versus Grok.

### 6.6 Token Budget Stress

Gemini, Grok, and Qwen remain structurally successful at all tested budgets, including 35%. Azure GPT instant is fragile under aggressive budgets: success is 47.9% at 35%, 77.1% at 50%, 95.8% at 75%, and 97.9% at 100%.

# 7 Controlled Follow-up: EXP06

EXP05 establishes a broad multilingual and multi-model signal: compressed protocols are a strong baseline, while hybrid_state improves state preservation. EXP06 tests this claim under stricter paired controls. The experiment removes natural and hybrid_min, directly compares compressed against hybrid_state, and adds a Chinese tokenizer/variant ablation plus three-judge calibration.

## 7.1 Design

EXP06 contains two generation phases and one evaluation phase. EXP06-A evaluates causal handoff transfer across ES, EN, and ZH. EXP06-B evaluates ZH, ZH_PINYIN, EN_LITERAL, and ES_LITERAL variants to probe the language/tokenization hypothesis. EXP06-C evaluates every valid generation with Gemini 3.5 Flash, Azure GPT 5.4 High, and Grok 4.20 Reasoning. The cleaned dataset contains 1,563 valid generations and 4,689 evaluations; 21 Azure GPT instant generations were retained as terminal policy failures and excluded from scoring.

## 7.2 Mode Results

Compressed achieves the higher global quality index in EXP06 (Table 4). Hybrid_state is weaker on global quality, semantic fidelity, operational continuity, compactness, ambiguity, and information loss. However, it is stronger on state preservation. This is the central controlled result: EXP06 confirms hybrid_state as a state-transfer specialization, not as a universal quality winner.

| Mode | $n$ | $Q$ | Fidelity | State | Continuity | Compact | Ambig. | InfoLoss |
|---|---|---|---|---|---|---|---|---|
| compressed | 777 | 3.855 | 3.780 | 3.728 | 3.761 | 4.705 | 2.231 | 2.498 |
| hybrid_state | 786 | 3.661 | 3.621 | 3.805 | 3.699 | 4.638 | 2.678 | 2.642 |

Table 4: EXP06 mode-level results. Higher is better for $Q$, fidelity, state, continuity, and compactness. Lower is better for ambiguity and information loss.

## 7.3 Paired Effects

Table 5 reports paired hybrid_state minus compressed deltas over matched experimental cells. Hybrid_state is lower on global quality ($\Delta Q = -0.208$, 95% CI $[-0.239, -0.177]$), but higher on state preservation ($\Delta = +0.066$, 95% CI $[+0.032, +0.100]$). This makes the EXP05 claim sharper: hybrid_state helps the operational-state variable, while compressed remains the safer global baseline.

## 7.4 Tokenizer/Variant Ablation

In the tokenizer ablation, native ZH remains stronger than romanized ZH_PINYIN (Table 6). This supports the hypothesis that script representation and tokenization interact with compact protocol behavior. It does not fully isolate tokenization from training distribution or language familiarity, but it is stronger evidence than the correlational ZH signal in EXP05.

## 7.5 Judge Calibration

Judge agreement is high for an LLM-as-judge setup (Table 7). Pairwise quality-index correlations range from 0.886 to 0.913. Azure GPT 5.4 High is stricter on average, Gemini 3.5 Flash is more

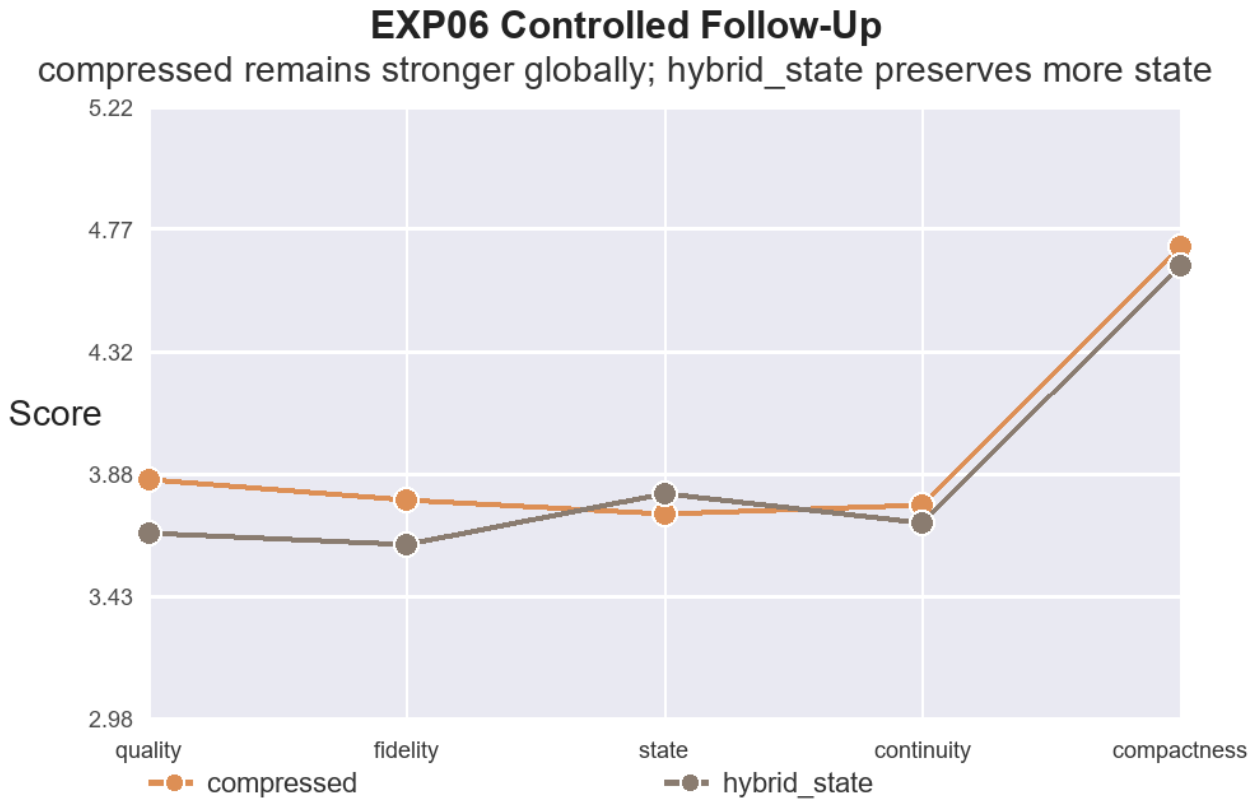


Figure 2: EXP06 controlled comparison between compressed and hybrid_state. The figure highlights the central tradeoff: compressed remains stronger globally, while hybrid_state preserves slightly more state.

generous, and Grok 4.20 Reasoning is intermediate. Because the same high-level conclusion survives three judges, EXP06 is less vulnerable to single-judge drift than EXP05 alone.

### 7.6 Interpretation

EXP06 strengthens the paper by converting EXP05's broad exploratory signal into a controlled follow-up. The result is not that hybrid_state dominates compressed. Instead, compressed remains the general-purpose baseline, while hybrid_state selectively improves the operational-state variable that motivated the protocol. This distinction makes the contribution sharper: symbolic protocols should be optimized and evaluated against their intended transfer function, not only against aggregate quality.

| Metric | $n$ pairs | $\Delta$ | 95% CI Low | 95% CI High |
|---|---|---|---|---|
| quality index | 774 | -0.208 | -0.239 | -0.177 |
| semantic fidelity | 774 | -0.174 | -0.210 | -0.138 |
| state preservation | 774 | +0.066 | +0.032 | +0.100 |
| operational continuity | 774 | -0.078 | -0.112 | -0.043 |
| handoff quality | 774 | -0.199 | -0.235 | -0.162 |
| compactness | 774 | -0.072 | -0.118 | -0.026 |
| ambiguity | 774 | +0.463 | +0.417 | +0.510 |
| information loss | 774 | +0.159 | +0.121 | +0.197 |

Table 5: EXP06 paired deltas for hybrid_state minus compressed. Positive deltas are beneficial for positive metrics but harmful for ambiguity and information loss.

| Variant | $n$ | $Q$ | Fidelity | State | Stability | Compact |
|---|---|---|---|---|---|---|
| ZH | 456 | 3.824 | 3.785 | 3.836 | 4.417 | 4.703 |
| ZH_PINYIN | 180 | 3.328 | 3.246 | 3.363 | 3.687 | 4.569 |
| EN_LITERAL | 180 | 3.595 | 3.506 | 3.650 | 4.341 | 4.617 |
| ES_LITERAL | 180 | 3.671 | 3.624 | 3.661 | 4.157 | 4.709 |

Table 6: EXP06 language/tokenizer ablation. Native ZH outperforms romanized ZH_PINYIN, suggesting that representation matters for compact protocol behavior.

# 8 Real-Agent Handoff Evaluation: EXP07

EXP07 tests whether the protocol family can be evaluated in real agent frameworks using objective continuation metrics. This experiment is smaller than EXP05/EXP06, but it changes the evidence type: outputs are scored for explicit operational recovery rather than only by LLM judges. The cleaned dataset contains 270 successful executions: 180 LangGraph executions and 90 OpenFang executions.

## 8.1 Objective Metrics

Each EXP07 task defines a handoff state containing variables, constraints, completed subtasks, pending subtasks, plan steps, risks, validation conditions, and the next action. The continuation agent must return strict JSON. We then compute:

- variable recovery rate;
- subtask completion rate;
- plan continuity rate;
- constraint retention rate;
- binary handoff success;
- state-error count.

These metrics are intentionally narrower than EXP05/EXP06's judge scores. They do not measure all semantic quality, but they provide a more objective view of whether operational state survived handoff.

## 8.2 Mode Results

Table 8 reports global EXP07 results by mode. Compressed remains the strongest operational baseline on variable recovery, plan continuity, and state-error count. Hybrid_state is slightly higher

| Judge A | Judge B | $n$ | Pearson $r$ | Mean Abs. $\Delta$ |
|---|---|---|---|---|
| Azure GPT 5.4 High | Gemini 3.5 Flash | 1561 | 0.901 | 0.570 |
| Azure GPT 5.4 High | Grok 4.20 Reasoning | 1561 | 0.913 | 0.395 |
| Gemini 3.5 Flash | Grok 4.20 Reasoning | 1563 | 0.886 | 0.512 |

Table 7: EXP06 judge agreement over quality index.

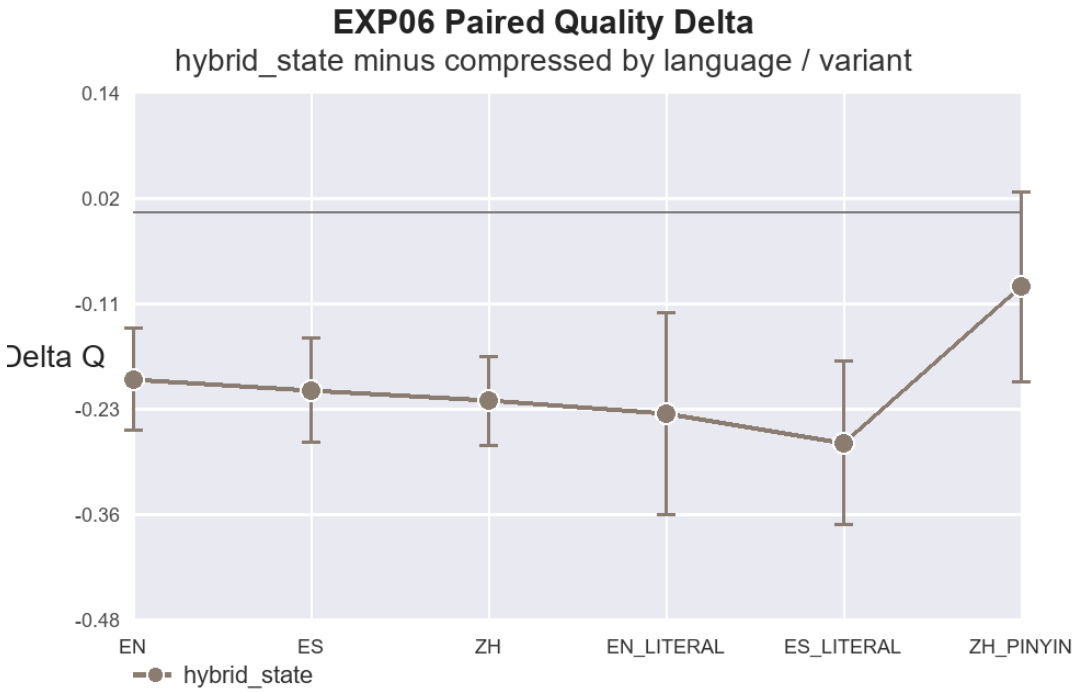


Figure 3: EXP06 judge-agreement matrix over quality index. Correlations are high enough to reduce single-judge risk, but not high enough to treat any judge as ground truth.

on constraint retention, but it does not win globally. Natural prose remains competitive on subtask completion and handoff success, but it produces more state errors.

### 8.3 Framework and Model Effects

Table 9 shows that framework surface matters. OpenFang reaches perfect scores under the current schema, while LangGraph exposes substantial variation across modes and models. We interpret the OpenFang result conservatively: it demonstrates compatibility between the protocol/task schema and an external real-agent surface, but should not be read as a universal ranking of OpenFang over LangGraph.

| Mode | $n$ | VarRec | Subtasks | PlanCont | Constraints | Handoff | StateErr |
|---|---|---|---|---|---|---|---|
| natural | 90 | 0.739 | 0.473 | 0.497 | 0.683 | 0.367 | 16.978 |
| compressed | 90 | 0.856 | 0.467 | 0.589 | 0.739 | 0.356 | 10.589 |
| hybrid_state | 90 | 0.750 | 0.364 | 0.442 | 0.744 | 0.344 | 10.811 |

Table 8: EXP07 real-agent objective metrics by mode. Higher is better except for state-error count.

| Framework | $n$ | VarRec | Subtasks | PlanCont | Constraints | Handoff | StateErr |
|---|---|---|---|---|---|---|---|
| LangGraph | 180 | 0.672 | 0.152 | 0.264 | 0.583 | 0.033 | 19.189 |
| OpenFang | 90 | 1.000 | 1.000 | 1.000 | 1.000 | 1.000 | 0.000 |

Table 9: EXP07 objective metrics by real-agent framework. OpenFang saturation should be interpreted as surface/schema compatibility, not as a general framework benchmark.

### 8.4 Paired Real-Agent Deltas

Table 10 reports paired hybrid_state minus compressed deltas. Within LangGraph, hybrid_state does not improve global operational recovery. With GPT chat latest, it improves constraint retention ($\Delta = +0.183$) and slightly reduces state errors ($\Delta = -0.133$), but loses subtask completion and plan continuity. With GPT 5.4 High, it is lower on most objective metrics. OpenFang is tied because all modes saturate the schema.

| Framework | Model | $n$ | Δ VarRec | Δ Subtasks | Δ Plan | Δ Constraints | Δ StateErr |
|---|---|---|---|---|---|---|---|
| LangGraph | GPT 5.4 High | 30 | -0.293 | -0.125 | -0.342 | -0.167 | +0.800 |
| LangGraph | GPT chat latest | 30 | -0.025 | -0.183 | -0.100 | +0.183 | -0.133 |
| OpenFang | GPT chat latest | 30 | 0.000 | 0.000 | 0.000 | 0.000 | 0.000 |

Table 10: EXP07 paired deltas for hybrid_state minus compressed. Positive values are beneficial except for state-error count, where lower is better.

### 8.5 Interpretation

EXP07 supports the broader research program but narrows the claim. It shows that compact handoff protocols can be exercised in real agent frameworks and scored with objective state-continuation metrics. However, it does not replicate the stronger judge-based hybrid_state advantage as a global real-agent effect. The most conservative interpretation is that compressed remains the robust baseline, while hybrid_state's state benefit is conditional: it may help explicit constraint retention in some real-agent/model strata, but it is not a universal operational winner.

## 9 Real-Agent Scale-Up: EXP08

EXP08 tests whether the EXP07 real-agent result survives a larger matrix. It uses 30 operational tasks, 2 protocol modes, 5 repetitions, and 3 framework/model routes: LangGraph with Azure GPT chat latest, LangGraph with Azure GPT 5.4 High, and OpenFang with Azure GPT chat latest. The operational-complete view contains 900/900 OK cells with no latest operational errors. Because strict JSON is part of the objective metric pipeline, we also report a stricter view containing 824/900 JSON-valid cells.

| View / Mode | $n$ | VarRec | Subtasks | PlanCont | Constraints | Handoff | StateErr |
|---|---|---|---|---|---|---|---|
| Operational / compressed | 450 | 0.894 | 0.105 | 0.526 | 0.813 | 0.033 | 5.478 |
| Operational / hybrid_state | 450 | 0.756 | 0.053 | 0.493 | 0.750 | 0.020 | 14.344 |
| JSON-valid / compressed | 432 | 0.932 | 0.109 | 0.548 | 0.847 | 0.035 | 1.581 |
| JSON-valid / hybrid_state | 392 | 0.867 | 0.061 | 0.566 | 0.861 | 0.023 | 1.819 |

Table 11: EXP08 real-agent scale-up by analysis view and mode. Higher is better except for state-error count.

In the operational-complete view, compressed is the stronger global baseline across variable recovery, subtask completion, plan continuity, handoff success, and state-error count. In the stricter JSON-valid view, hybrid_state is slightly higher on constraint retention and plan continuity, but it still does not win globally. The most important new observation is a failure mode rather than a protocol win: Azure GPT 5.4 High under LangGraph produced 76 blank-output cells with `parse_error=no_json_object`, including 58 in hybrid_state and 18 in compressed. These calls are operationally completed but not usable for strict schema scoring, showing that high-reasoning model routes can fail under JSON-constrained handoff even when the surrounding agent runner succeeds.

## 10 Real Tool-Use Validation: EXP09

EXP09 tests the next step after real-agent handoff: whether a protocol-conditioned agent can complete deterministic file-level tasks through a tool interface. The experiment uses 24 controlled repository-like cases, 2 modes, 3 repetitions, and 2 routes. The `reference` route is a deterministic local tool agent used to validate the task bank and scoring harness. The `gemini_3_5_flash` route asks Gemini 3.5 Flash to emit tool actions in strict JSON. Tools include file listing, file reading, file writing, CSV writing, checksum computation, JSONL validation, security sweep, and validator execution.

| Route | Mode | OK | Total | Success |
|---|---|---|---|---|
| reference | compressed | 72 | 72 | 1.000 |
| reference | hybrid_state | 72 | 72 | 1.000 |
| Gemini 3.5 Flash | compressed | 72 | 72 | 1.000 |
| Gemini 3.5 Flash | hybrid_state | 72 | 72 | 1.000 |

Table 12: EXP09 latest successful tool-use cells by route and mode. Historical repaired errors are preserved separately.

| Task family | Gemini OK | Gemini Total |
|---|---|---|
| read_file | 36 | 36 |
| modify_csv | 30 | 30 |
| json_validation | 30 | 30 |
| table_generation | 24 | 24 |
| manifest | 24 | 24 |

Table 13: EXP09 Gemini 3.5 Flash latest cells by deterministic task family.

The final latest-cell view is saturated: 288/288 cells pass their deterministic validators. This

should not be read as evidence that the task is intrinsically easy or that all protocol interfaces are robust. The raw log contains 155 repaired historical errors. These failures exposed interface problems that are scientifically relevant: models emitted action schemas using `type` rather than `action`, returned direct action lists rather than an `actions` object, wrote `{path,text}` without a tool name, attempted to invent checksums instead of invoking a checksum tool, and sometimes exhausted output under excessive thinking. After the runner accepted common schema variants, added a deterministic `compute_hashes` tool, and reduced Gemini's thinking budget, the same matrix completed. EXP09 therefore supports a methodological claim: the protocol/tool interface can support deterministic file-level task completion under objective validators, but interface contracts and reasoning budgets are part of the experimental system.

| Experiment | Final latest OK | Repaired errors | Dominant repaired failure modes |
|---|---|---|---|
| EXP09 | 288/288 | 155 | Tool-action schema variants, missing tool names, checksum hallucination, JSON envelope mismatch, excessive thinking budget. |
| EXP10 | 80/80 | 17 | Parser/tool-contract mismatch, unknown tool names, missing files during early attempts, claim-copying over forbidden claims. |

Table 14: Historical repaired interface failures for saturated tool-agent experiments. Final success rates should be read together with these repaired-error logs.

## 11 Public Repo Tool-Agent Culmination: EXP10

EXP10 turns the public research artifact itself into the task surface. Instead of asking an agent to solve isolated synthetic file tasks, the agent must maintain a landing/repository package that summarizes the experimental path, preserves the paper's allowed claims, avoids forbidden overclaims, updates dependent artifacts consistently, and passes deterministic validators. This is still a controlled benchmark, not arbitrary software engineering: tasks are scoped, validators are known, and the tool contract is explicit. The security validator checks for credential-like strings, bearer-token patterns, suspicious key files, and direct secret leakage in generated public files.

| Route | Mode | OK | Total | Success |
|---|---|---|---|---|
| dry-local scaffold | compressed | 16 | 16 | 1.000 |
| Gemini 3.5 Flash | compressed | 32 | 32 | 1.000 |
| Gemini 3.5 Flash | hybrid_state | 32 | 32 | 1.000 |

Table 15: EXP10 latest successful public-repository tool-agent cells. The Gemini matrix completed 64/64 cells.

The `dry-local scaffold` row is a deterministic harness sanity check. It does not call a model and does not test protocol sensitivity; it is logged under `compressed` only to avoid double-counting a mode-invariant local baseline. The protocol comparison in EXP10 is therefore the 64-cell Gemini matrix.

The final EXP10 view is saturated: 80/80 latest cells succeed, including 64/64 Gemini cells. This is a useful culmination because it evaluates the protocol on the public artifact that explains the research itself. It is also a limitation. A saturated result cannot distinguish protocols by aggregate

| Metric | Mean | Interpretation |
|---|---|---|
| claim_drift_rate | 0.000 | No validated mutation of allowed/forbidden scientific claims. |
| scope_drift_rate | 0.000 | No unexpected file modifications in latest successful cells. |
| dependency_preservation_score | 0.763 | Most declared artifact dependencies were preserved. |
| recovery_score | 0.878 | Handoff recovery remained high after repaired interface failures. |
| constraint_preservation_score | 1.000 | Critical constraints were preserved in latest successful cells. |

Table 16: EXP10 objective metrics over latest successful cells. Higher is better except for drift rates.

success, and the task set remains controlled. The raw log contains 17 repaired historical interface errors, mostly parser/tool-contract failures and claim-copying behavior. EXP10 therefore supports a reproducibility and systems claim, not a broad benchmark claim: under a clear parser, validator, and tool contract, a protocol-conditioned tool agent can maintain a public research package while preserving claims, scope, dependencies, and recovery behavior.

## 12 Real Multi-Agent Scientific Repository Benchmark: EXP13

EXP13 extends the public-artifact line into a more discriminative benchmark. Instead of a single tool-agent surface, each scenario is framed as a role-conditioned scientific-repository maintenance task with writer, reviewer, reproducibility, security/claims, and merge/repair responsibilities. The benchmark contains 16 scenarios, two modes (`compressed` and `hybrid_state`), two model routes (Gemini 3.5 Flash and Azure GPT-5.4 High), and two repetitions, for 128 real-model cells. Validators check merge safety, constraint preservation, dependency preservation, claim drift, scope drift, regression rate, repair success, secret safety, local-path leakage, responsive behavior, and HTML integrity.

| Condition | Latest cells | Success | Failure |
|---|---|---|---|
| `compressed` | 64 | 64 | 0 |
| `hybrid_state` | 64 | 62 | 2 |
| Gemini 3.5 Flash | 64 | 63 | 1 |
| Azure GPT-5.4 High | 64 | 63 | 1 |

Table 17: EXP13 latest-cell summary. Unlike EXP10, the benchmark is not fully saturated; two final failures are retained.

The two retained failures are informative. Gemini 3.5 Flash failed one `hybrid_state` outreach scenario with `no_json_object`, a machine-readable output contract failure. Azure GPT-5.4 High failed one `hybrid_state` role scenario with `no_changes`, an action failure in which no applicable edit was produced. This result strengthens the paper's conservative interpretation: `compressed` remains the strongest operational baseline, while `hybrid_state` can expose contract fragility in longer role-conditioned tasks even when it is valuable for explicit state representation.

## 13 Experimental History and Design Rationale

The final paper is best understood as a sequence of experiments rather than one monolithic benchmark. Earlier experiments established the motivation and vocabulary. EXP05 discovered a broad

cross-model signal. EXP06 tested the central causal contrast under tighter controls. EXP07 and EXP08 moved the evaluation into real-agent surfaces. EXP09 and EXP10 tested deterministic tool and repository interfaces.

### 13.1 From EXP01 to EXP04

EXP01–EXP04 were preparatory stages. They established that ordinary natural summaries and ad hoc compression were insufficient for durable handoff. The early work identified three recurring failure types: loss of operational variables, mutation of constraints, and fluent continuation without plan continuity. These failures motivated the later separation between semantic quality and state preservation.

EXP04 is especially important historically because it supplied the winning compact hybrid family that later became the `hybrid_min` and `hybrid_state` distinction. The lesson was that extreme brevity alone is not enough. A protocol must preserve the information that a later agent needs to act.

### 13.2 EXP05 as Broad Discovery

EXP05 is deliberately ambitious. It crosses languages, modes, models, judges, and mini-experiments. Its purpose is not perfect causal isolation; its purpose is signal discovery. The result is clean enough to justify a paper: `compressed` is a strong universal baseline, `hybrid_state` improves state preservation in paired analysis, `hybrid_min` is too lossy for primary use, and ZH behavior is unusually strong.

EXP05 also establishes a systems lesson. Provider quotas, JSON parsing, fallback judges, and model route differences are not peripheral. They shape what kinds of protocols can be run at scale. The experiment therefore led directly to resumability, ledgers, stricter no-fallback policies, and better judge calibration.

### 13.3 Mini-Experiments Inside EXP05

MiniEXP-A compared Grok reasoning and non-reasoning behavior. MiniEXP-B measured judge drift. MiniEXP-C tested token-budget stress. MiniEXP-D focused on Chinese compactness and representation. MiniEXP-E tested long multi-agent handoff. These mini-experiments are not the central proof, but they guide interpretation. They show where protocol behavior changes: evaluator family, reasoning mode, token budget, language representation, and handoff length.

### 13.4 EXP06 as Controlled Follow-Up

EXP06 narrows the research question. Instead of comparing all modes, it tests the strongest claim: `hybrid_state` should preserve state better than `compressed`. The result is exactly the kind of result that survives review better than a simple win: `hybrid_state` improves state preservation but loses global quality. This produces a meaningful tradeoff rather than a marketing claim.

EXP06 also introduces the Chinese representation ablation. Native ZH remains stronger than ZH_PINYIN, but the paper avoids claiming full tokenizer causality. The result is evidence that representation matters, not proof of one mechanism.

### 13.5 EXP07 and EXP08 as Real-Agent Validation

EXP07 moves from judge scores to real-agent objective metrics. EXP08 scales this design. These experiments are crucial because they prevent the paper from depending only on LLM-as-judge scores. They also discipline the claims. In real-agent settings, `compressed` remains the stronger aggregate baseline. `hybrid_state` has narrower advantages, mainly around selected state and constraint dimensions.

This matters scientifically. If a protocol improves state preservation under judges but not every real-agent metric, the honest conclusion is not failure. The honest conclusion is that protocol value is surface-dependent.

### 13.6 EXP09 and EXP10 as Interface Culmination

EXP09 and EXP10 are not broad model benchmarks. They are interface-validity experiments. EXP09 checks whether protocol-conditioned agents can complete deterministic file-level tasks. EXP10 checks whether a public research artifact can be maintained without claim drift, scope drift, or dependency loss. Their final success rates are high, even saturated, so they do not discriminate protocol quality strongly. Their value lies in showing that the protocol program can be connected to real tools and validators.

The repaired errors are the most important part of EXP09 and EXP10. Schema mismatch, parser failure, checksum hallucination, missing tool names, and output-budget behavior are exactly the kinds of interface failures that break real agents. Therefore these experiments should be read as systems lessons that support the practical side of the paper.

## 14 Cost, Pricing, and Quota History

Cost is part of the experimental system. A protocol that reduces tokens but causes retries, parse failures, or blank outputs may be more expensive operationally than a longer protocol that completes reliably. The experiments therefore maintain cost ledgers and preserve failure events.

### 14.1 Why Estimated Cost Is Used

Provider billing dashboards are delayed and sometimes aggregated. Experimental control requires immediate estimates after each call. The cost ledger records estimated input tokens, output tokens, model route, stage, and estimated price. These estimates are not final invoices. They are per-call experimental telemetry.

### 14.2 Observed Cost Behavior

EXP05 demonstrated that a large multilingual multi-model benchmark can be run surprisingly cheaply when prompts are compact and outputs are bounded. Mid-run estimates were close to dashboard-observed spend, within an acceptable margin. Later experiments showed a different pattern: high-reasoning or high-output routes can accumulate cost through blank outputs, retries, or long JSON responses even when the number of successful cells is moderate.

### 14.3 Quota as Experimental Constraint

Quota shaped scheduling. Gemini evaluation required rate control during constrained windows. Azure routes had different practical limits and billing visibility. AI Studio and Vertex routes be-

haved differently with respect to available quota and credential style. These are not only operational annoyances; they affect reproducibility. A future rerun should record provider, model route, date, region, quota policy, and retry schedule.

### 14.4 Cost Versus Scientific Value

The paper does not optimize only for cheapest output. Some expensive calls are valuable if they provide judge calibration, frontier-model comparison, or failure-mode evidence. However, cost must not contaminate the experiment. For that reason, model substitution is explicitly marked, evaluator fallback is avoided in primary scoring unless declared, and high-cost auditors are reserved for selected analyses.

| Stage | Cost role | Risk | Mitigation |
|---|---|---|---|
| EXP05 | broad discovery | quota and evaluator fallback | resumable parts and ledger |
| EXP06 | controlled confirmation | policy failures and judge variance | paired design and three judges |
| EXP07 | objective handoff | framework saturation | separate framework summaries |
| EXP08 | scale-up | blank strict-JSON outputs | operational and strict views |
| EXP09 | tool use | parser/schema repair | historical error log |
| EXP10 | public artifact | saturated success | drift/dependency metrics |

Table 18: Cost and quota risks by experimental stage.

### 14.5 Pricing History as Reproducibility Metadata

The model market changes quickly. Prices, quotas, and model availability can change between paper writing and rerun. Therefore pricing should be treated as metadata. The reproducibility package should include the observed model route, date, estimated unit prices used by the ledger, and dashboard-observed final costs where available. This prevents a future reader from confusing a 2026 provider surface with a stable benchmark object.

### 14.6 Interpretation of Cheap Runs

Cheap successful runs are scientifically useful but can be misleading. A low cost can mean the protocol is efficient. It can also mean outputs are too short, evaluators are too permissive, or failures are not being counted. The ledger and historical-error logs are therefore paired: cost without error history is incomplete.

## 15 Claim Boundaries and Strength

The strongest claim is that compressed is the most reliable general-purpose baseline in this experimental family. It wins quality in EXP05, remains ahead in EXP06 under stricter paired controls, and is also the best aggregate operational baseline in EXP07 and EXP08 real-agent objective metrics.

The second strong claim is narrower: hybrid_state improves state preservation under judge-scored paired designs. This appears in EXP05's principal paired analysis and survives EXP06's stricter paired design. The claim should not be overstated as a global quality victory or a universal real-agent operational win.

The third claim is methodological: real-agent handoff and tool use can be measured with objective validators. EXP07 demonstrates objective state-continuation metrics in LangGraph and OpenFang, EXP08 scales the real-agent setting to 900 operational executions, EXP09 shows deterministic file-level tool-use completion over controlled repository tasks, and EXP10 applies the same idea to the public research artifact itself. The substantive real-agent result remains mixed: compressed is stronger globally in EXP07/EXP08, while hybrid_state shows narrower constraint/plan benefits only in selected strata. EXP09 and EXP10 are best read as successful protocol-interface validations rather than new global protocol rankings.

The ZH/tokenization claim is promising but still partly exploratory. EXP05 shows a ZH advantage, and EXP06-B shows native ZH outperforming romanized ZH_PINYIN. This supports a representation effect, but not a complete causal explanation.

The weakest claims are any broad model-intelligence rankings. Generator effects in this paper reflect protocol/task/provider behavior, not general model capability.

## 16 Discussion

The main result is not that a symbolic protocol simply beats natural language everywhere. Instead, EXP05, EXP06, EXP07, EXP08, EXP09, and EXP10 show tradeoffs between global quality, compactness, operational state, real-agent continuation, tool-interface reliability, and public-artifact maintenance. Compressed remains the strongest universal baseline: it performs well across languages, models, controlled follow-up tests, and real-agent objective metrics. Hybrid_state gives up some global quality but improves state preservation in judge-based paired experiments, which is precisely the property that motivated the protocol. EXP07 and EXP08 add a caution: the hybrid_state advantage should not be generalized to every real-agent metric or framework. EXP09 and EXP10 add a second caution: a protocol can only succeed operationally if the action schema, available tools, parser, dependency model, validator contract, and output-budget settings are aligned with the task.

This suggests a practical design rule: compact agent protocols should be optimized by target function. If the goal is general semantic quality, compressed is the safer baseline. If the goal is durable handoff and preservation of operational state, hybrid_state is more promising. If the goal is extreme brevity, hybrid_min is compact but risky.

The ZH result is one of the most interesting signals. Chinese performs best in the EXP05 principal language comparison and remains stronger than ZH_PINYIN in EXP06-B. A plausible explanation is that script density, tokenizer behavior, and model training distribution interact with symbolic compression. However, even EXP06 cannot fully separate these factors. The safest claim is that representation matters; the exact mechanism remains open.

Judge drift also matters. The same outputs can receive systematically different quality levels depending on evaluator family. This does not invalidate the result, but it means future papers should report judge identity, judge agreement, and sensitivity analysis.

## 17 Limitations

EXP05 is exploratory and systems-oriented, while EXP06 is a narrower controlled follow-up but still depends on LLM judges and provider serving layers. EXP07 and EXP08 reduce judge dependence through objective metrics, but their framework surfaces are not interchangeable. EXP09 and EXP10 further reduce judge dependence, but they use controlled repository-like tasks rather than a large live codebase. The use of LLM judges introduces evaluator bias, even though EXP06 reduces

single-judge risk through three-judge calibration. Some models were accessed through provider-specific serving layers, so observed behavior includes provider routing, quota, truncation, latency, safety filtering, and content-policy behavior. In EXP06, 21 Azure GPT instant generations were retained as terminal policy failures and excluded from score aggregation; therefore Azure generator averages should be read with survivorship bias in mind. In EXP08, 76 Azure GPT 5.4 High cells returned blank outputs under strict JSON scoring; these are retained as a model/serving failure mode rather than silently removed. In EXP09, 155 historical errors were repaired by improving the action parser, checksum tool contract, and Gemini thinking budget; these are preserved as interface-failure evidence and should not be hidden behind the final 288/288 latest-cell success rate. In EXP10, 17 historical errors were repaired before the final 80/80 latest-cell view; the saturated result should be interpreted as a controlled public-artifact interface validation, not evidence that general repository maintenance is solved.

The quality index is descriptive, not a validated psychometric scale. It is useful for ranking and paired contrasts, but the paper's main scientific claims rely on individual metrics such as state preservation, operational continuity, compactness, ambiguity, and information loss. The ZH effect is promising but not fully causal. EXP06-B strengthens the representation hypothesis by comparing ZH with ZH_PINYIN and literal variants, but it still cannot isolate tokenizer mechanics from training distribution and language familiarity. Some mini-experiments use smaller samples and should be treated as hypothesis-generating.

The model list is also time-sensitive. Frontier model names, availability, quotas, and pricing change rapidly. Therefore, this paper should describe exact observed model identifiers, dates, and serving routes in an appendix before publication. EXP07's OpenFang results saturate the objective schema, and EXP08's OpenFang results remain high, so they should be interpreted as successful compatibility checks rather than general framework benchmarks. EXP09 and EXP10 saturated final results should likewise be interpreted as controlled protocol/tool-interface validations, not as evidence that arbitrary repository work is solved.

## 18 Conclusion

EXP05 through EXP13 together reframe prompt compression as interoperable state-transfer protocol design. EXP05 provides the broad discovery result: compressed is a strong cross-model, multilingual baseline, and hybrid_state improves state preservation in paired analysis. EXP06 provides the controlled follow-up: compressed remains ahead on global quality, but hybrid_state still improves state preservation under stricter paired controls. EXP07 adds real-agent validation, and EXP08 scales it: compact protocols can be executed and measured in LangGraph and OpenFang using objective handoff metrics, but hybrid_state does not dominate globally in this setting. EXP09 shows that the same research program can be connected to deterministic file-level tool use, where validator contracts and tool schemas become part of the protocol surface. EXP10–EXP12 extend that validation to the public research artifact itself, measuring claim drift, scope drift, dependency preservation, recovery, responsive behavior, and conflict-aware maintenance under controlled repository/page-maintenance task sets. EXP13 provides the clearest repository benchmark result so far: compressed was more operationally robust (64/64) than hybrid_state (62/64), while hybrid_state exposed contract fragility in longer role-conditioned tasks. The conclusion is therefore not that one protocol dominates all uses. Rather, compressed is the universal baseline, and hybrid_state is a specialized state-oriented protocol whose benefits are conditional and should be tested against the target execution surface.

This distinction makes the research program clearer. Future compact agent protocols should be

evaluated by target function: general semantic quality, state preservation, continuity, recoverability, cost, framework behavior, and robustness to language and model family. The results motivate further work on symbolic protocols for multi-agent AI systems, especially where agents must survive context loss, provider substitution, multilingual transfer, evaluator drift, and real-agent execution constraints.

# A Appendix A: Protocol Templates

This appendix gives auditable templates for the four protocol modes. The exact task banks and generated instances are part of the reproducibility package.

## A.1 Natural Template

```
Continue the task using the following context.
Preserve the current facts, constraints, and next steps.
Do not introduce unsupported claims.
Return the requested artifact in the target language.
```

## A.2 Compressed Template

```
goal=<task goal>; state=<known facts>; vars=<variables>;
constraints=<must/forbid>; evidence=<supporting observations>;
next=<next action>; risk=<known failure mode>.
```

## A.3 Hybrid-Min Template

```
g=<goal>; st=<state>; v=<vars>; c=<constraints>;
e=<evidence>; n=<next>; r=<risk>.
```

## A.4 Hybrid-State Template

```
HYBRID_STATE{
  GOAL: <task goal>;
  STATE: <facts and variables>;
  CONSTRAINTS: <allowed and forbidden claims/actions>;
  PLAN: <ordered continuation steps>;
  DEPENDENCIES: <files, figures, tables, prior outputs>;
  NEXT: <single immediate next action>;
  RISKS: <known drift/truncation/parser risks>
}
```

# B Appendix B: Evaluator JSON Schema

```
{
  "semantic_fidelity": 1-5,
  "clarity": 1-5,
  "utility": 1-5,
  "completeness": 1-5,
```

```
  "state_preservation": 1-5,
  "operational_continuity": 1-5,
  "context_recoverability": 1-5,
  "handoff_quality": 1-5,
  "compactness": 1-5,
  "ambiguity": 1-5,
  "information_loss": 1-5,
  "rationale": "short non-scoring explanation"
}
```

# C Appendix C: Objective Validator Metrics

Objective metrics are intentionally narrower than judge metrics. `variable_recovery` measures whether required variables appear correctly in the continuation. `subtask_completion` measures whether the expected next actions are completed. `plan_continuity` measures whether the agent continues the correct plan rather than starting a new one. `constraint_retention` measures whether allowed and forbidden claims remain intact. `handoff_success` measures whether the next agent can act from the transferred state. `state_error_count` counts explicit state mistakes.

# D Appendix D: Data-Cleaning Algorithm

1. Parse all JSONL run events.
2. Compute a stable cell identifier from phase, task, language or variant, mode, model, judge, framework, and repetition.
3. Sort events by timestamp or append order.
4. Select the latest successful event per cell for scored analysis.
5. Retain terminal policy failures separately when no successful event exists.
6. Retain historical parse errors, quota events, blank outputs, and repaired interface failures in audit logs.
7. Compute aggregates only from scored latest-cell records unless a table explicitly reports historical failures.

# E Appendix E: Repaired Error Taxonomy

Historical repaired errors are grouped into quota failures, parse failures, schema mismatches, missing tool names, checksum hallucination, blank output, policy refusal, and scope/dependency drift. The paper reports these because final clean success rates can hide the actual engineering difficulty. EXP09 and EXP10 especially depend on this taxonomy: their final success is only meaningful when paired with the repair history.

# F Appendix F: Model Route Metadata

Each scored run should be accompanied by provider name, model identifier, date, region where applicable, access surface, role, and notes. The following roles recur across the experimental program: generator, evaluator, auditor, tool agent, deterministic scaffold, and local validator. Hosted model names are time-sensitive and should not be treated as immutable scientific objects.

# G Appendix G: Reproducibility Checklist

1. Include exact prompt templates and task banks.
2. Include cleaned latest-cell datasets.
3. Include historical error logs.
4. Include cost ledgers and pricing assumptions.
5. Include freeze manifests and checksums.
6. Include analysis scripts.
7. Include figure-generation scripts.
8. Exclude secrets, provider keys, private logs, and unredacted local paths.
9. Record model routes, dates, providers, and regions.
10. State which results are exploratory, controlled, objective, or interface-validating.

# H Appendix H: Claim Strength Table

| Claim class | Interpretation |
|---|---|
| Strong | `compressed` is the strongest universal baseline across this experimental family. |
| Strong but narrow | `hybrid_state` improves judge-scored state preservation in paired designs. |
| Bounded | Real-agent objective metrics are feasible and expose different tradeoffs from LLM judges. |
| Exploratory | Native ZH and ZH_PINYIN differences suggest representation effects. |
| Interface-validating | EXP09/EXP10 show deterministic protocol/tool contracts can support controlled file-level tasks. |
| Non-claim | The paper does not claim that `hybrid_state` wins global quality or that any model ranking measures general intelligence. |

# I Appendix I: Experiment-by-Experiment Audit Notes

**EXP05 audit note.** EXP05 serves as broad multilingual multi-model discovery. Its main lesson is that `compressed` is robust; `hybrid_state` preserves state; ZH is promising. The experiment is therefore interpreted within its own surface rather than forced into a single global ranking. For reproducibility, the corresponding freeze should preserve task templates, prompts, model routes, run events, cost ledger entries, cleaned latest-cell records, and repaired-error logs.

**EXP06 audit note.** EXP06 serves as controlled paired confirmation. Its main lesson is that `hybrid_state` improves state but loses global quality. The experiment is therefore interpreted within its own surface rather than forced into a single global ranking. For reproducibility, the corresponding freeze should preserve task templates, prompts, model routes, run events, cost ledger entries, cleaned latest-cell records, and repaired-error logs.

**EXP07 audit note.** EXP07 serves as real-agent objective handoff. Its main lesson is that objective metrics can disagree with judge scores. The experiment is therefore interpreted within its

own surface rather than forced into a single global ranking. For reproducibility, the corresponding freeze should preserve task templates, prompts, model routes, run events, cost ledger entries, cleaned latest-cell records, and repaired-error logs.

**EXP08 audit note.** EXP08 serves as real-agent scale-up. Its main lesson is that operational and strict-JSON views must be separated. The experiment is therefore interpreted within its own surface rather than forced into a single global ranking. For reproducibility, the corresponding freeze should preserve task templates, prompts, model routes, run events, cost ledger entries, cleaned latest-cell records, and repaired-error logs.

**EXP09 audit note.** EXP09 serves as deterministic file-level tool use. Its main lesson is that parser and validator contracts are part of the protocol. The experiment is therefore interpreted within its own surface rather than forced into a single global ranking. For reproducibility, the corresponding freeze should preserve task templates, prompts, model routes, run events, cost ledger entries, cleaned latest-cell records, and repaired-error logs.

**EXP10 audit note.** EXP10 serves as public repository maintenance. Its main lesson is that claim drift, scope drift, dependency preservation, and recovery are measurable. The experiment is therefore interpreted within its own surface rather than forced into a single global ranking. For reproducibility, the corresponding freeze should preserve task templates, prompts, model routes, run events, cost ledger entries, cleaned latest-cell records, and repaired-error logs.

# J Appendix J: Metric Definitions and Measurement Notes

The following table expands the metric vocabulary used throughout the experiments. The main paper reports aggregate results, but reproducibility requires knowing what each score attempts to measure.

| Metric | Measurement intent |
|---|---|
| `semantic_fidelity` | Whether the continuation preserves the meaning of the original state and avoids unsupported semantic mutation. |
| `clarity` | Whether a downstream reader or agent can interpret the transferred state without unnecessary ambiguity. |
| `utility` | Whether the output helps the next step of the task rather than merely restating context. |
| `completeness` | Whether required facts, constraints, and deliverables are present. |
| `state_preservation` | Whether operational variables, status flags, constraints, and partial work are retained. |
| `operational_continuity` | Whether the output makes it possible to continue the same work without restarting the plan. |
| `context_recoverability` | Whether a new agent can recover the prior context from the protocol alone. |
| `handoff_quality` | Whether the transferred representation is suitable as a handoff object between agents. |
| `compactness` | Whether the protocol achieves information density without deleting critical state. |

| Metric | Measurement intent |
|---|---|
| `ambiguity` | Whether multiple incompatible interpretations remain possible; lower is better. |
| `information_loss` | Whether important content disappears; lower is better. |

# K Appendix K: Failure Case Studies

The most useful failures in this project are not spectacular model mistakes. They are small interface or state-transfer errors that would silently damage a long-running agent workflow. The following cases are representative of the failure classes preserved in the repaired-error logs.

**Constraint mutation.** A receiving model changes a bounded claim into a stronger claim, such as turning state-specific advantage into global superiority. Mitigation: Represent allowed and forbidden claims explicitly and validate claim drift.

**Variable loss.** A model preserves the topic but drops an operational variable required for continuation. Mitigation: Score variable recovery and use fielded state blocks.

**Plan restart.** The next agent writes a fresh plan instead of continuing the existing plan. Mitigation: Score plan continuity and include NEXT plus PLAN fields.

**Parser noncompliance.** The model produces useful prose but invalid JSON or invalid tool calls. Mitigation: Use strict schemas, retry policy, and historical parse-error logs.

**Scope drift.** A tool agent edits files outside the requested surface. Mitigation: Validate expected files and `unexpected_files_touched`.

**Dependency loss.** An agent edits a file but forgets related figures, manifests, checksums, or indexes. Mitigation: Track `dependency_preservation_score`.

**Blank output.** A high-reasoning or strict-output route returns an empty result under budget pressure. Mitigation: Separate operational-complete and strict-JSON-valid views.

**Quota interruption.** Provider windows stop the run mid-cell. Mitigation: Use resumable append-only runners and wait/retry policies.

# L Appendix L: Reading Guide for the Figures

The plots in this report are intentionally line-oriented rather than decorative. The goal is to show tradeoff shape. When two curves cross, the reader should not ask only which protocol is higher on average; the reader should ask which function the protocol is optimizing. For example, a protocol can be lower on global quality but higher on state preservation. That is not contradiction; it is the core phenomenon.

### L.1 EXP05 Figure Interpretation

The EXP05 mode plot should be read as a discovery profile. `compressed` has the best general shape. `hybrid_state` is close enough to be scientifically interesting and has the expected state-preservation advantage. `hybrid_min` shows the danger of optimizing only for brevity. `natural` remains useful but does not automatically preserve operational state better than structured compression.

### L.2 EXP06 Figure Interpretation

The EXP06 figure is stricter. It shows that `hybrid_state` is not a global quality winner. This is important because it protects the paper against overclaiming. A weaker but sharper claim is more valuable: `hybrid_state` moves state preservation in the intended direction under paired controls.

### L.3 EXP07 and EXP08 Figure Interpretation

The real-agent plots shift the interpretation from judge preference to continuation behavior. They show that operational execution has its own constraints. A protocol that helps state preservation in judge scoring can still lose on variable recovery or plan continuity in a framework-specific validator. This is a systems result, not a failure of the research question.

## M Appendix M: Cost Ledger Fields

The cost ledger should contain enough information to reconstruct the economic and operational trace of the run. Recommended fields include: timestamp, experiment, phase, cell id, provider, model id, route, stage, prompt token estimate, completion token estimate, reasoning token estimate when available, unit price assumption, estimated cost, cumulative estimated cost, status, error class, retry count, and dashboard reconciliation note.

| Field family | Examples | Why it matters |
|---|---|---|
| Identity | experiment, phase, cell id | joins run, score, and cost data |
| Provider | provider, route, region | model behavior is route-dependent |
| Tokens | input, output, reasoning | explains price and truncation risk |
| Status | ok, quota, parse, blank | separates science from operations |
| Accounting | unit price, estimate, cumulative | supports budget control |
| Audit | retry count, reconciliation | preserves reproducibility |

Table 21: Recommended cost-ledger field families.

## N Appendix N: Expanded Release Plan

The public package should be layered. The first layer is the paper and figures. The second layer is cleaned aggregate CSV and JSONL latest-cell data. The third layer is redacted historical logs. The fourth layer is scripts for cleaning, analysis, and figure generation. The fifth layer is a manifest with checksums. Raw private logs and credentials should never be released.

### N.1 Safe Public Release

A safe release contains no API keys, no bearer tokens, no credential files, no unredacted local paths, and no private provider console traces. It should include enough data for a reviewer to recalculate

the main tables. It should also include representative prompt instances and evaluator schemas. A release that contains only the final PDF is not reproducible enough for this paper's claims.

### N.2 Audit Samples

At minimum, the release should include a sample of good outputs, bad outputs, repaired failures, and ambiguous judge cases. The bad outputs are scientifically important because they show the boundary of the protocol. The repaired failures are practically important because they show which interface contracts were necessary for deterministic tool success.

## O Appendix O: Expanded Threats to Validity

### O.1 Construct Validity

State preservation is not identical to human usefulness. The metric captures operational variables and constraints, but a human team may value additional dimensions such as explanatory style, domain correctness, or ease of inspection. The paper therefore avoids claiming that one protocol is universally best for all collaboration.

### O.2 Internal Validity

Provider routing, quota windows, retries, and output budgets can affect results. The append-only log and latest-cell cleaning reduce but do not eliminate these risks. EXP06 improves internal validity through paired design, but judge-based scoring remains an instrument with bias.

### O.3 External Validity

The task banks are designed around research and agent handoff, not every possible software or scientific workflow. EXP09 and EXP10 use controlled repository-like tasks; they do not prove arbitrary repository maintenance. The strongest external claim is methodological: the protocol family can be evaluated with objective validators.

### O.4 Statistical Conclusion Validity

The current confidence intervals are useful but not sufficient for final causal certainty across all strata. A stronger analysis should use hierarchical models or clustered bootstrap procedures. The paper declares this instead of hiding the limitation behind many tables.

## P Appendix P: Practical Protocol Selection Guide

If the goal is general answer quality, use `compressed`. If the goal is explicit state preservation, test `hybrid_state`. If the goal is extreme token reduction, `hybrid_min` is useful only as a stress condition. If the goal is human interpretability, `natural` may remain suitable. If the goal is tool use, protocol choice is secondary to schema clarity, parser tolerance, validator determinism, and explicit file scope.

# Q Appendix Q: Reviewer-Facing Answers

**Why so many experiments?** The experiments form a chain: broad discovery, controlled confirmation, real-agent objective validation, tool-use validation, and public-artifact culmination. The chain is long because state transfer is a systems problem.

**Why not claim hybrid_state wins?** Because it does not win globally. It improves state preservation under judge-scored paired designs and shows conditional advantages only in selected operational strata.

**Why include EXP09 and EXP10 if saturated?** Because they test interface validity, not global protocol superiority. Their repaired failures reveal schema and parser lessons.

**Why use LLM judges at all?** Some semantic dimensions are hard to validate deterministically. The paper reduces judge dependence through three-judge calibration and later objective validators.

**What would make the result stronger?** Public data release, mixed-effect modeling, tokenizer-level audits, larger real-agent task banks, and live repository tasks with independent validators.

# R Appendix R: Protocol Cards for Replication

This appendix rewrites the protocol family as practical replication cards. The goal is to make the formats inspectable without requiring the reader to infer them from aggregate results.

## R.1 Natural Card

Use `natural` when human readability is the priority and when the downstream agent is expected to infer state from prose. The card should include: task goal, current state, known constraints, evidence, next action, and forbidden overclaims. The risk is that prose can hide structure. A sentence can mention a constraint without making it operationally binding. Therefore natural handoff should be considered a strong readability baseline but a weak state-binding protocol.

## R.2 Compressed Card

Use `compressed` as the default operational baseline. It should use short fields but avoid excessive symbolic density. The minimum fields are goal, state, constraints, evidence, next, and risk. The reason this baseline performs well is that it keeps enough natural language for model robustness while making the most important fields visible. The main risk is that it may still blend state, plan, and constraints into one compact clause.

## R.3 Hybrid-Min Card

Use `hybrid_min` only as a stress condition or when token budget is extremely constrained. It compresses field names and content aggressively. It is useful scientifically because it tests the lower bound of symbolic compression. It is risky operationally because small omissions can become large handoff failures.

## R.4 Hybrid-State Card

Use `hybrid_state` when state preservation matters more than global fluency. The card should expose goal, state, variables, constraints, evidence, plan, dependencies, next action, and risks. Its benefit is not that it sounds better. Its benefit is that it gives the receiving agent a structured map of what must survive. The main risk is parser burden and possible output rigidity.

## R.5 Card Comparison

| Mode | Best use | Main risk | Expected role |
|---|---|---|---|
| `natural` | human inspection | implicit state loss | prose baseline |
| `compressed` | general handoff | mild field blending | universal baseline |
| `hybrid_min` | token stress | information loss | lower-bound test |
| `hybrid_state` | state transfer | parser and rigidity cost | state-specialized protocol |

# S Appendix S: Audit Trail and Chain of Evidence

The chain of evidence has four levels. Level one is the raw append-only event log. Level two is the cleaned latest-cell dataset. Level three is the aggregate analysis table. Level four is the paper claim. A reproducible paper must allow a reviewer to move backward from level four to level one.

## S.1 From Log to Claim

A claim such as "`hybrid_state` improves state preservation in EXP06" should be traceable. The reviewer should be able to identify paired cells, compute state-preservation deltas, verify confidence intervals, check which judges contributed, and inspect whether any errors were excluded. If a cell was retried, the historical record should show the earlier failure and the later success.

## S.2 Why Historical Errors Matter

Historical errors are not simply noise. They show the operational cost of a protocol. A mode that produces more parse failures may look acceptable after cleaning but remain expensive to operate. A model that returns blank outputs under strict JSON scoring may have strong reasoning ability but weak interface reliability. A tool agent that succeeds after parser repair may demonstrate the importance of the contract rather than the strength of the model alone.

## S.3 Audit Trail Checklist

1. Can every table row be traced to a clean dataset file?
2. Can every clean row be traced to an append-only event?
3. Are superseded failures retained?
4. Are terminal policy failures separated from scored outputs?
5. Are provider model routes and dates recorded?
6. Are prompt templates and evaluator schemas available?
7. Are figure scripts deterministic?
8. Are secrets removed from public artifacts?

# T Appendix T: Expanded Result Interpretation

This appendix gives a slower reading of the major result families. It is written for readers who want the conceptual story more than the compact table summary.

## T.1 Result Family 1: Baseline Robustness

The most stable result is that `compressed` is hard to beat. This matters because the baseline is not weak. A new protocol is scientifically interesting only if it beats or complements a serious baseline. The paper therefore does not frame `compressed` as a straw man. It is the method a practical agent engineer would probably choose first: compact, readable, and tolerant of model variation.

## T.2 Result Family 2: State Preservation Tradeoff

`hybrid_state` repeatedly moves state preservation upward in judge-scored paired designs. It also loses global quality under stricter EXP06 controls. This is the central tradeoff. A protocol can be worse as general prose and still better as an operational state container. This is the reason the paper reframes prompt compression as protocol design.

## T.3 Result Family 3: Language Representation

The ZH signal is strong enough to motivate future work but not strong enough to close the mechanism. Native ZH performing better than ZH_PINYIN suggests that representation matters. However, representation includes more than tokenizer length. It includes training distribution, script familiarity, and model-specific multilingual behavior.

## T.4 Result Family 4: Agent Surface Dependence

EXP07 and EXP08 show that real-agent handoff is not identical to judge-scored handoff. A judge may reward explicit state fields, while a tool or framework validator may reward direct variable recovery or plan execution. This means protocol choice should be conditioned on the execution surface.

## T.5 Result Family 5: Interface Repair

EXP09 and EXP10 are most valuable as interface-repair studies. Their final success rates are high, but the path matters. Parser improvements, schema clarity, checksum tools, and output-budget tuning changed the system. This supports the claim that protocol design must include machine-readable contracts.

# U Appendix U: Replication Narrative

A future replication should begin with EXP06 rather than EXP05. EXP06 is narrower and easier to reproduce. The replicator should choose two modes, `compressed` and `hybrid_state`, one or two generator models, at least two judges, and a small language set. The first target should be paired state-preservation deltas. If that result replicates, the replicator can add EXP07-style objective handoff metrics.

## U.1 Minimal Replication

The minimal replication contains 6 tasks, 2 modes, 2 languages, 2 repetitions, 1 generator, and 2 judges. This is not enough for the full paper claim, but it can test the basic tradeoff: quality versus state preservation.

## U.2 Medium Replication

The medium replication contains 15 tasks, 2 modes, 3 languages or variants, 2 repetitions, 3 generators, and 3 judges. This approximates the controlled part of the paper while remaining cheaper than the full EXP05 discovery run.

## U.3 Full Replication

The full replication should include discovery, controlled follow-up, objective handoff, and tool-use validation. It should preserve all logs, compute cost ledgers, and publish a redacted reproducibility package. The result should not be judged only by whether the exact numbers match. The stronger question is whether the same tradeoff shape appears: `compressed` robust globally, `hybrid_state` useful for explicit state, and real-agent execution dependent on interface contracts.

## U.4 Expected Failure Modes in Replication

The most likely replication failures are provider drift, model route changes, judge score calibration shifts, stricter safety filtering, token-budget differences, JSON parser differences, and task-bank ambiguity. These should be treated as data, not only as obstacles. If a protocol fails because a parser contract is too brittle, that is directly relevant to the research question.

# V Appendix V: Extended Methodology Summary

The methodological core can be summarized as follows. First, define protocol modes by their intended transfer function. Second, instantiate task banks that require continuation rather than isolated answer generation. Third, run models through append-only resumable execution. Fourth, evaluate with both LLM judges and objective validators. Fifth, clean with latest-cell scoring while retaining historical errors. Sixth, report cost, quota, and interface failures as part of the system. Seventh, draw claims only at the strength supported by the evidence.

This sequence is intentionally conservative. It prevents a compact prompt from being declared superior simply because it is shorter, cheaper, or favored by one judge. It also prevents a failed parser event from being dismissed as irrelevant. In agent systems, parser events are part of reality.

# W Appendix W: Limitations Matrix

The limitations are easier to audit when grouped by source. Model limitations include route drift, changing model names, unknown provider-side updates, safety filters, and quota windows. Evaluation limitations include judge bias, JSON parser brittleness, score calibration drift, and the descriptive nature of the quality index. Task limitations include controlled handoff templates, limited live-world diversity, and possible overfitting to research workflows. Tool limitations include deterministic validators that may miss qualitative errors and controlled repositories that do not fully represent large production codebases.

| Risk source | Example | Declared mitigation |
|---|---|---|
| Model route | hosted route changes | record provider, date, and model identifier |
| Evaluator | judge score drift | three-judge calibration and objective validators |
| Task bank | controlled templates | describe scope and avoid broad benchmark claims |
| Tool interface | parser contract failure | preserve repaired-error logs |
| Cost | delayed dashboard billing | keep per-call estimated ledger |
| Statistics | correlated cells | recommend clustered bootstrap or mixed models |

# X Appendix X: Publication Plan

The report can be published in two forms. The first is a compact workshop paper focusing on EXP05 through EXP08, with EXP09 and EXP10 moved to appendix. The second is this long technical report, where the full experimental program is visible. The workshop version should emphasize the main scientific result: `compressed` is the universal baseline and `hybrid_state` is a state-specialized protocol. The long report should emphasize reproducibility, audit trails, cost history, and interface lessons.

For arXiv, the long report is acceptable if the title makes clear that it is a technical report. For a venue submission, the paper should probably be shortened and split. Paper A would cover multilingual handoff and state preservation. Paper B would cover deterministic tool-use interfaces and repository maintenance. This split would reduce cognitive load for reviewers.

# Y Appendix Y: Figure Expansion Plan

The current paper includes the most important figures, but a full visual appendix should include more. Recommended additional plots are: cost versus quality, state preservation versus compactness, paired deltas by language, paired deltas by generator, judge agreement heatmaps, blank-output rates by model and mode, strict JSON validity by framework, and repaired-error counts by error class. The goal is not decoration. The goal is to let the reader see tradeoff shape rather than only table values.

The style should remain simple: grey background, white grid, line plots for metric profiles, bar plots for counts, and paired-delta plots for controlled comparisons. Each figure should answer one question. A figure that only repeats a table should be removed or moved to appendix.

# Z Appendix Z: Final Release Checklist

Before public release, run the following checklist. Compile the PDF. Extract text and inspect for encoding artifacts. Confirm that every figure renders. Run a secret sweep. Confirm that public data contains no API keys, bearer tokens, credential files, or private local paths. Confirm that prompts, validators, schemas, task banks, clean datasets, repaired-error logs, analysis scripts, and checksums are present. Confirm that model routes and dates are listed. Confirm that claim-strength language is conservative. Confirm that EXP09 and EXP10 are described as interface validations rather than broad proof of general repository-solving ability.

The final public package should be boring in the best sense: no hidden dependencies, no mystery scripts, no private credentials, and no unsupported claims. The science is strongest when a skeptical reader can reproduce the tables and see exactly where the system failed before it was repaired.

## Data and Code Availability

We release the prompts, task templates, validators, schemas, cleaned latest-cell datasets, repaired-error logs, analysis scripts, freeze manifests, and checksum manifests used for EXP05–EXP13 where public release is safe. The release excludes API keys, credential files, private provider logs, and unredacted local paths. Exact artifact locations are listed in the reproducibility appendix and public repository.

## References


[1] Anonymous Authors. EXP06 Causal Protocol Transfer: Frozen Data, Cleaned Dataset, and Statistical Reports. Internal experimental dataset freeze, May 26, 2026.

[2] Anonymous Authors. EXP05 Premium Multi-Model Trilingual: Cleaned Experimental Data and Reports. Internal experimental dataset freeze, May 23, 2026.

[3] Anonymous Authors. EXP07 Real-Agent Handoff Objective Metrics: Frozen Data, Cleaned Dataset, and Statistical Reports. Internal experimental dataset freeze, May 26, 2026.

[4] Anonymous Authors. EXP08 Real-Agent Scale-Up: Frozen Data, Objective Metrics, and Blank-Output Failure Analysis. Internal experimental dataset freeze, May 27, 2026.

[5] Anonymous Authors. EXP09 Real Tool-Use Agent Tasks: Frozen Data, Cleaned Dataset, and Deterministic Validator Reports. Internal experimental dataset freeze, May 27, 2026.

[6] Anonymous Authors. EXP10 Public Repo Tool-Agent: Frozen Data, Public Landing Artifact, and Deterministic Validator Reports. Internal experimental dataset freeze, May 27, 2026.

[7] Jason Wei et al. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. NeurIPS 2022. arXiv:2201.11903. https://arxiv.org/abs/2201.11903.

[8] Shunyu Yao et al. ReAct: Synergizing Reasoning and Acting in Language Models. ICLR 2023. arXiv:2210.03629. https://arxiv.org/abs/2210.03629.

[9] Nelson F. Liu et al. Lost in the Middle: How Language Models Use Long Contexts. TACL 2024. arXiv:2307.03172. DOI: 10.48550/arXiv.2307.03172. https://arxiv.org/abs/2307.03172.

[10] Yupeng Chang et al. A Survey on Evaluation of Large Language Models. ACM Transactions on Intelligent Systems and Technology, 15(3), 2024. DOI: 10.1145/3641289. arXiv:2307.03109. https://doi.org/10.1145/3641289. https://arxiv.org/abs/2307.03109.